

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский
политехнический университет»**

Электротехнический факультет

Выпускающая кафедра: информационные технологии и автоматизированные
системы (ИТАС)

Направление подготовки: 09.03.04 – Программная инженерия

**ОТЧЕТ
ПО ТВОРЧЕСКОЙ РАБОТЕ**

На тему:

**«Автоматизация рабочего места хозяина по отслеживанию состояния
питомца»**

Выполнили студент группы РИС-25-26

Виноградов Сергей Валерьевич,

Пермь 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
РАЗДЕЛ 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	5
1.1 Зоотехнические нормы содержания домашних животных	5
1.2 Физические принципы измерения массы и передачи	7
1.3 Анализ существующих программно-аппаратных решений	8
РАЗДЕЛ 2. ПРАКТИЧЕСКАЯ ЧАСТЬ	11
2.1 Архитектура приложения (UML-диаграммы)	11
2.1.1 Диаграмма классов	11
2.1.2 Диаграмма последовательности	21
2.2 Реализация графического интерфейса	23
2.3 Реализация модуля ежедневного трекинга.....	26
2.4 Реализация аппаратного модуля и связи с ним.....	30
2.5 Реализация и визуализация	34
ЗАКЛЮЧЕНИЕ	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	38

ВВЕДЕНИЕ

В современном мире почти в каждой семье есть домашние питомцы, для которых требуются постоянный уход и большое внимание к их здоровью, питанию и общему благополучию. Однако в повседневной суете владелец может забыть, сколько воды выпила кошка за день или насколько изменился вес собаки за месяц. Бумажные записи теряются, заметки в телефоне не систематизированы, а точные расчеты норм требуют времени и специальных данных.

Особенно остро стоит вопрос контроля питания: хозяину тяжело отследить, поел ли питомец или нет, потому что он может быть занят работой, домашними делами или спать. Питомец может подойти к миске в любое время дня и ночи, и в результате владелец не знает, съел ли тот положенную порцию сегодня, сколько раз подходил к миске, не остался ли голодным, выпил ли обязательную норму воды за день, не переедает ли.

В связи с этим возникает необходимость в создании автоматизированного приложения, которое будет автоматически контролировать изменение веса миски с кормом и водой и передавать владельцу простой отчет со статистикой съеденного корма и выпитой воды.

Целью проекта является разработка автоматизированного рабочего места хозяина по уходу за домашним питомцем, которое обеспечивает ведение карточек питомцев, ежедневный учет потребления воды и корма, контроль соответствия нормам, а также интеграцию с аппаратным модулем на базе Arduino для автоматического сбора весовых данных.

Задачи проекта:

1. Провести анализ предметной области и существующих решений для учета здоровья домашних животных.
2. Обосновать выбор технологий (C++, Arduino, Qt) для реализации АРМ.

3. Спроектировать архитектуру приложения с использованием сигнально-слотовой модели Qt.
4. Реализовать графический интерфейс, включающий все требуемые виджеты.
5. Разработать модуль ежедневного трекинга и алгоритмы расчета норм воды и веса.
6. Создать аппаратный модуль на Arduino с двумя тензодатчиками и Bluetooth.
7. Обеспечить интеграцию между АРМ и аппаратным модулем.
8. Выполнить модульное тестирование ключевых алгоритмов.

РАЗДЕЛ 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Зоотехнические нормы содержания домашних животных

Для правильного ухода за домашними животными необходимо учитывать особенности их видов и пород. В ветеринарии и зоотехнии разработаны рекомендации по кормлению, поению и контролю веса домашних животных.

Вода является важным питательным веществом, поскольку она отвечает за поддержание множества физиологических функций и является средой для удаления метаболических отходов. Водная составляющая тела кошки или собаки находится в постоянном движении, поэтому регулирование водного баланса и потребление воды, вызванное жаждой, необходимы для восполнения постоянных потерь воды в процессе испарения через дыхание и кожные поверхности. [1] Дефицит жидкости сказывается очень плохо на здоровье питомца. Обезвоживание быстро нарушает обмен веществ, снижает уровень энергии и может привести к серьезным последствиям, включая почечную недостаточность. Поддержание водного баланса – основа здоровья животного. [2]

Суточная потребность животного в воде зависит от массы тела. Для собак рекомендуется 40-60 миллилитров воды на килограмм веса в сутки. Для кошек – 30-50 миллилитров на килограмм веса в сутки. Эти значения являются средними и могут корректироваться в зависимости от активности питомца, температуры окружающей среды и типа кормления. При кормлении сухим кормом потребность в воде возрастает, при кормлении влажным – снижается.

Правильное питание является одним из фундаментальных столпов здоровья, долголетия и благополучия питомца. Однако, несмотря на кажущуюся простоту, определение точной суточной нормы корма – это задача, требующая внимательности и учета множества индивидуальных факторов. Недостаточное или избыточное кормление может привести к серьезным проблемам: недоедание – к дефициту питательных веществ, потере

веса, задержке развития; переедание – к ожирению, которое, в свою очередь, является фактором риска для сахарного диабета, заболеваний суставов, сердца и других хронических патологий.

Большинство владельцев ориентируются на рекомендации, указанные на упаковке корма, но эти данные являются лишь общими ориентирами. Понимание ключевых факторов, влияющих на энергетические потребности животного, знание методов расчёта суточной нормы и осознание важности постоянного мониторинга являются фундаментальными для каждого ответственного владельца.

Энергетические потребности кошки или собаки не являются постоянной величиной и зависят от сложного взаимодействия множества внутренних и внешних факторов:

1. **Вид животного.** Кошки являются облигатными хищниками, их метаболизм настроен на получение энергии преимущественно из белков и жиров животного происхождения. Они имеют более высокие потребности в белке и таурине. Собаки более гибки в своём рационе, будучи факультативными хищниками, и могут эффективно использовать углеводы как источник энергии.
2. **Возраст.** Котята и щенки находятся в периоде интенсивного роста и развития, поэтому их энергетические потребности на килограмм массы тела значительно выше, чем у взрослых животных. Им требуется больше калорий, белка, витаминов и минералов для формирования костей, мышц и органов. Взрослые животные имеют стабилизированные энергетические потребности. Пожилые животные имеют замедленный метаболизм и сниженный уровень активности, поэтому им требуется меньше калорий, чтобы предотвратить набор лишнего веса.
3. **Вес и размер (порода).** Более крупной собаке или кошке потребуется больше корма, чем более мелкой. Однако, чем крупнее животное, тем меньше калорий ему требуется на каждый килограмм массы тела. То

есть крупный пёс весом 40 килограммов будет потреблять меньше калорий на килограмм, чем мелкая собака весом 5 килограммов. Породные особенности влияют на уровень метаболизма и предрасположенность к тем или иным заболеваниям.

4. **Уровень активности.** Малоактивные домашние животные требуют меньше калорий. Активные, спортивные или рабочие собаки нуждаются в значительно большем количестве энергии.
5. **Физиологическое состояние.** После стерилизации или кастрации метаболизм замедляется, а аппетит часто повышается. Таким животным требуется значительно меньше калорий для поддержания веса. Беременным и кормящим самкам требуются очень высокие нормы калорий. При наличии хронических заболеваний энергетические потребности могут меняться, и требуется специализированная диета. [3]

1.2 Физические принципы измерения массы и передачи

Для автоматического контроля веса миски с кормом и водой используются тензодатчики. Тензодатчик – это датчик, основанный на эффекте тензорезистивности: при деформации проводника его электрическое сопротивление изменяется. Когда на тензодатчик кладут груз, он деформируется, и его сопротивление меняется пропорционально приложенной массе. [4]

Однако изменение сопротивления очень мало, поэтому его нельзя измерить напрямую аналоговым входом микроконтроллера. Для этого используется усилитель НХ711. НХ711 содержит 24-битный аналого-цифровой преобразователь (АЦП), который усиливает сигнал с тензодатчика и преобразует его в цифровое значение. Высокая разрядность АЦП (24 бита) обеспечивает точность измерения до долей грамма. [5]

Для перевода цифрового значения АЦП в граммы используется калибровочный коэффициент. Процесс калибровки заключается в следующем:

на датчик помещают эталонный груз известной массы, считывают цифровое значение с НХ711, затем вычисляют коэффициент по формуле:

$$k = V / m \quad (1)$$

где k – калибровочный коэффициент (единицы АЦП на грамм); V – цифровое значение, считанное с АЦП; m – масса эталонного груза, г.

В разрабатываемом устройстве калибровочные коэффициенты были подобраны экспериментально. Для датчика под миской с кормом коэффициент составил 176. Для датчика под миской с водой — 140.

Для передачи данных с Arduino на компьютер используется Bluetooth-модуль HC-05. HC-05 работает в режиме прозрачной передачи данных, то есть он принимает байты через последовательный порт (UART) и отправляет их по Bluetooth, а полученные по Bluetooth байты выводит в последовательный порт. Для Arduino это выглядит как обычное соединение с последовательным портом, но вместо проводов используется радиоканал. Протокол RFCOMM, используемый HC-05, эмулирует последовательный порт по Bluetooth.

1.3 Анализ существующих программно-аппаратных решений

На сегодняшний день существуют различные способы контроля питания домашних животных, которые можно разделить на три категории.

1. **Ручные способы.** К ним относятся визуальное наблюдение, бумажные журналы и заметки в телефоне. Владелец самостоятельно фиксирует, когда и сколько, по его мнению, съел питомец. Основной недостаток – низкая точность и неполнота данных. В отсутствии хозяина данные о кормлении не фиксируются.

2. **Программные решения с ручным вводом.** Существуют мобильные приложения, которые позволяют вести учет питания, веса и вакцинаций. Они предоставляют удобный интерфейс и возможность построения графиков, но по-прежнему требуют, чтобы владелец самостоятельно вносил каждую запись. Проблема незафиксированных эпизодов в отсутствие хозяина не решается.
3. **Аппаратные решения с автоматической фиксацией.** Умные миски (PetKit, SureFeed и другие) оснащены встроенными весами и датчиками присутствия животного. Они автоматически определяют, когда питомец подходит к миске, фиксируют объём съеденного и передают данные в приложение. Однако такие устройства имеют два существенных недостатка. Первый — высокая стоимость: цена умной миски начинается от десяти тысяч рублей. Второй — привязка к собственной экосистеме: данные нельзя легко экспортировать или интегрировать с другими приложениями.

Учитывая высокую стоимость готовых устройств и необходимость в индивидуальной настройке, целесообразно разработать собственное решение на базе Arduino. Arduino — это открытая платформа для прототипирования, которая отличается низкой ценой, доступностью компонентов и простотой программирования. Общая стоимость компонентов разрабатываемого модуля (Arduino Nano, два тензодатчика, два усилителя HX711, Bluetooth HC-05) составляет около двух тысяч рублей, что в пять раз дешевле умной миски. При этом функциональность остается аналогичной: автоматическое взвешивание и передача данных на компьютер.

Вывод по разделу 1. В теоретической части рассмотрены зоотехнические нормы содержания домашних животных: суточные нормы воды и веса для собак и кошек, классификация пород. Описаны физические принципы работы тензодатчиков, усилителя HX711 и Bluetooth-модуля HC-05. Проведен анализ существующих решений, показано, что ручные способы не

обеспечивают полноты данных, а готовые умные миски имеют высокую стоимость. Обоснована целесообразность самостоятельной разработки на базе Arduino.

2.1 Архитектура приложения (UML-диаграммы)

2.1.1 Диаграмма классов

[illegible]

11

Пояснение диаграммы классов:

1. Главный класс MainWindow. Класс MainWindow является окном приложения. Он наследуется от QMainWindow – базового класса Qt для главных окон, который предоставляет готовую структуру с центральным виджетом, панелями инструментов, меню и док-областями.

Атрибуты MainWindow:

- `ui : Ui::MainWindow*` — указатель на объект пользовательского интерфейса, сгенерированный Qt Designer. Содержит все визуальные компоненты: кнопки, поля ввода, таблицы. Вынесен в отдельную структуру Ui для разделения логики и представления.
- `speciesBreeds : QMap<QString, QMap<QString, BreedNorm>>` — вложенный словарь, хранящий нормы для пород. Ключ первого уровня — вид животного (кошка/собака). Ключ второго уровня — порода на английском языке. Значение — структура BreedNorm с нормами корма, воды и веса. Такой контейнер выбран для быстрого доступа к нормам по виду и породе.
- `seriousDiseases : QSet<QString>` — множество серьёзных заболеваний. Qset обеспечивает быструю проверку наличия заболевания в списке. Используется при анализе состояния питомца.
- `minorDiseases : QSet<QString>` — множество незначительных заболеваний. Хранится отдельно от серьёзных, так как по-разному влияет на рекомендации по уходу.
- `m_sensor : SensorHardware*` — указатель на объект для работы с аппаратными датчиками. Хранится в главном окне, так как именно MainWindow управляет подключением и отображением данных.

- `m_tracker : DailyTracker*` — указатель на объект ежедневного трекера. Отвечает за учёт потребления воды и корма.
- `m_autoAnalyzeTimer : QTimer*` — таймер для автоматического анализа. Периодически запускает анализ собранных данных и показывает владельцу отклонения от норм.
- `m_sensorEnabled : bool` — флаг, включён ли режим работы с датчиками. Если выключен, программа использует ручной ввод.
- `m_manualInputMode : bool` — флаг ручного режима ввода. Когда датчики не подключены или отключены, владелец может вносить данные вручную.

Методы MainWindow:

- `MainWindow()` и `~MainWindow()` — конструктор и деструктор. В конструкторе создаются все виджеты, загружаются данные о породах и заболеваниях, настраиваются соединения сигналов и слотов. В деструкторе освобождаются ресурсы.
- `initData()` — загрузка справочных данных: пород с их нормами, списков заболеваний.
- `initSensors()` — инициализация подсистемы датчиков. Создаёт объект `SensorHardware` и подключает его сигналы к слотам `MainWindow`.
- `setTableEditMode(editable)` — включает или выключает режим редактирования таблицы с питомцами.
- `addLogMessage(message)` — добавляет сообщение в лог-панель для отладки и информирования владельца.
- `breedsForSpecies(speciesEng)` — возвращает список пород для указанного вида. Используется для заполнения `QcomboBox` при выборе вида животного.

- `getBreedNorm(speciesEng, breedEng)` — возвращает структуру `BreedNorm` для указанной породы. Необходима для расчёта норм корма и воды.
- `analyzeFiveDays(...)` — анализирует данные за последние пять дней. Принимает вид, породу, вес, возраст, список заболеваний и векторы с данными о еде и воде. Возвращает текстовый результат анализа с рекомендациями.
- `drawGraphs(...)` — строит графики потребления еды и воды. Принимает векторы фактических данных и значения норм.
- `showAnalysisDialog(...)` — создаёт и открывает диалог `AnalysisDialog`, передавая ему данные для отображения.

Слоты `MainWindow` (вызываются сигналом):

- `onSpeciesChanged(index)` — вызывается при выборе вида животного в `QcomboBox`. Обновляет список пород для выбранного вида.
- `onBreedChanged(index)` — вызывается при выборе породы. Отображает нормы для этой породы (вес, корм, вода).
- `onAnalyzeClicked()` — вызывается при нажатии кнопки «Анализ». Собирает данные за последние дни и запускает анализ.
- `onConnectSensor()` — вызывается при нажатии кнопки подключения датчиков. Устанавливает соединение с `Arduino` через последовательный порт.
- `onDisconnectSensor()` — отключает датчики.
- `onSensorDataReceived(water, food, total)` — вызывается при получении новых данных от `SensorHardware`. Передаёт текущий вес воды и корма в `DailyTracker`.

- `onConnectionStatus(connected, status)` — вызывается при изменении статуса подключения к датчикам. Обновляет индикатор в интерфейсе.
- `onSensorError(error)` — вызывается при возникновении ошибки в работе датчиков. Показывает сообщение владельцу.
- `onRawDataReceived(rawLine)` — вызывается при получении сырых данных от датчиков. Используется для отладки.
- `onRecordSaved(timestamp, food, water)` — вызывается при сохранении новой записи в `DailyTracker`. Обновляет интерфейс.
- `onHistoryUpdated(records)` — вызывается при обновлении истории записей. Передаёт актуальный список для отображения.
- `onConsumptionUpdated(food, water)` — вызывается при обновлении текущего потребления. Обновляет отображение съеденного и выпитого.
- `onAutoAnalyzeCheck()` — вызывается при изменении состояния чекбокса автоматического анализа. Включает или выключает таймер.
- `onTableCellChanged(row, column)` — вызывается при изменении данных в таблице питомцев.

Структура `BreedNorm`. Хранит нормы для конкретной породы:

- `nameRus : QString` — русское название породы.
- `avgFood : double` — средняя суточная норма корма в граммах.
- `avgWater : double` — средняя суточная норма воды в миллилитрах.
- `weightMin : double` — минимальный здоровый вес в килограммах.
- `weightMax : double` — максимальный здоровый вес в килограммах.

- heightMin : double — минимальная высота в холке в сантиметрах.
- heightMax : double — максимальная высота в холке в сантиметрах.

2. Класс DailyTracker и структура DailyRecord. Класс DailyTracker отвечает за ежедневный учёт потребления воды и корма. Он наследуется от QObject.

Атрибуты DailyTracker:

- m_saveTimer : QTimer* — таймер для автоматического сохранения. Периодически вызывает onSaveTimer().
- m_saveInterval : int — интервал автоматического сохранения в секундах.
- m_startWater : double — начальный вес воды в миске (в момент начала отслеживания).
- m_startFood : double — начальный вес корма в миске.
- m_currentWater : double — текущий вес воды.
- m_currentFood : double — текущий вес корма.
- m_consumedWater : double — общее количество выпитой воды с начала отслеживания.
- m_consumedFood : double — общее количество съеденного корма с начала отслеживания.
- m_initialized : bool — флаг, инициализирован ли трекер (начальные веса установлены).
- m_history : QVector<Record> — вектор записей (история). QVector выбран для эффективного хранения и доступа к записям по индексу.

Методы DailyTracker:

- DailyTracker(parent) — конструктор. Создаёт таймер сохранения.

- `setSaveInterval(seconds)` — устанавливает интервал автоматического сохранения.
- `startTracking()` — начинает отслеживание. Запоминает текущие веса как начальные. Обнуляет счётчики потребления.
- `stopTracking()` — останавливает отслеживание. Принудительно сохраняет текущие данные.
- `getStartWater()`, `getStartFood()` — возвращают начальные веса.
- `getConsumedWater()`, `getConsumedFood()` — возвращают общее потребление с начала отслеживания.
- `getLast5Records()` — возвращает последние пять записей из истории. Используется для анализа.
- `addManualRecord(timestamp, food, water)` — добавляет ручную запись (когда датчики отключены).

Слоты `DailyTracker`:

- `updateCurrentWeight(water_ml, food_g)` — обновляет текущие веса. Вычисляет разницу с предыдущим весом и добавляет её к потреблению. При значительном изменении (питомец поел или попил) запоминает событие.
- `forceSave()` — принудительно сохраняет текущую запись.
- `onSaveTimer()` — вызывается по таймеру. Сохраняет текущие показатели как запись.

Приватные методы `DailyTracker`:

- `saveCurrentRecord()` — сохраняет текущую запись (текущее потребление и временную метку) в историю.
- `updateHistory(record)` — обновляет историю и испускает сигнал `historyUpdated`.

Сигналы `DailyTracker`:

- recordSaved(timestamp, food, water) — испускается при сохранении записи.
- historyUpdated(records) — испускается при обновлении истории.
- consumptionUpdated(food_g, water_ml) — испускается при изменении потребления.

Структура Record (вложенная): хранит одну запись: временную метку, количество выпитой воды и количество съеденного корма.

3. Класс SensorHardware и связанные элементы. Класс SensorHardware отвечает за связь с аппаратным модулем на Arduino через последовательный порт. Наследуется от QObject и реализует интерфейс ISensorProvider.

Атрибуты SensorHardware:

- m_portName : QString — имя последовательного порта (например, COM3)
- m_hSerial : HANDLE — хендл открытого последовательного порта.
- m_running : atomic<bool> — атомарный флаг, работает ли поток чтения.
- m_connected : atomic<bool> — атомарный флаг, подключено ли устройство.
- m_hasData : atomic<bool> — атомарный флаг, есть ли новые данные.
- m_dataMutex : mutable Qmutex — мьютекс для синхронизации доступа к данным из разных потоков.
- m_waterWeight : double — последний полученный вес воды.
- m_foodWeight : double — последний полученный вес корма.
- m_totalWeight : double — последний полученный общий вес.

Методы SensorHardware:

- `SensorHardware(portName, parent)` — конструктор. Запоминает имя порта.
- `~SensorHardware()` — деструктор. Останавливает поток и закрывает порт.
- `start()` — запускает работу с датчиками. Открывает порт и запускает поток чтения. Возвращает `true` при успехе.
- `stop()` — останавливает работу. Завершает поток и закрывает порт.
- `isConnected()` — возвращает статус подключения.
- `portName()` — возвращает имя порта.
- `getWaterWeight()`, `getFoodWeight()`, `getTotalWeight()` — возвращают последние веса.
- `hasValidData()` — возвращает `true`, если есть хотя бы один валидный пакет данных.
- `sendCommand(cmd)` — отправляет команду на Arduino (например, для тарировки).
- `readLoop()` — приватный метод, выполняющийся в отдельном потоке. Постоянно читает данные из последовательного порта и вызывает `parseLine()`.
- `parseLine(line)` — разбирает строку, пришедшую с Arduino. Ожидает формат DATA: вес1, вес2, сумма. При успешном разборе обновляет веса и испускает сигнал `dataReceived`.
- `openSerial()` — открывает последовательный порт с параметрами 9600 бод, 8 бит, 1 стоп-бит, без чётности.
- `closeSerial()` — закрывает порт.

Сигналы `SensorHardware`:

- `dataReceived(water_g, food_g, total_g)` — испускается при получении валидной строки данных. Передаёт вес воды, вес корма и общий вес.

- `connectionStatusChanged(connected, status)` — испускается при изменении состояния подключения.
- `errorOccurred(error)` — испускается при ошибке (например, порт не открылся).
- `rawDataReceived(rawLine)` — испускается для каждой прочитанной строки (используется для отладки).

Структура `SensorData`: хранит один пакет данных от датчиков:

- `water_ml : double` — вес воды в миллилитрах.
- `food_g : double` — вес корма в граммах.
- `total_g : double` — общий вес.
- `valid : bool` — признак валидности данных.

Интерфейс `IsensorProvider`: определяет общий контракт для всех провайдеров данных:

- `GetLatestData() : SensorData` — возвращает последние данные.
- `IsConnected() const : bool` — возвращает статус подключения.
- `SendCommand(cmd : const std::string&) : bool` — отправляет команду.

Класс `SensorHardware` реализует этот интерфейс, что позволяет добавлять другие источники данных без изменения `MainWindow`.

4. Класс `AnalysisDialog`. Класс `AnalysisDialog` представляет диалоговое окно для отображения результатов анализа. Наследуется от `QDialog`.

Атрибуты `AnalysisDialog`:

- `ui : Ui::AnalysisDialog*` — указатель на объект пользовательского интерфейса.

Методы `AnalysisDialog`:

- `AnalysisDialog(parent)` — конструктор. Создаёт интерфейс.
- `~AnalysisDialog()` — деструктор.

- `setResult(resultText)` — устанавливает текстовый результат анализа (например, рекомендации для владельца).
- `setGraphData(foods, waters, foodNorm, waterNorm)` — передаёт данные для построения графиков.
- `setTitle(petInfo)` — устанавливает заголовок диалога (информация о питомце).
- `drawGraphs(foods, waters, foodNorm, waterNorm)` — приватный метод, строит графики. Сравнивает фактические данные с нормой.

2.1.2 Диаграмма последовательности

Ниже на рисунке 2 представлена диаграмма последовательности для сценария «Получение данных с датчиков и обновление интерфейса»

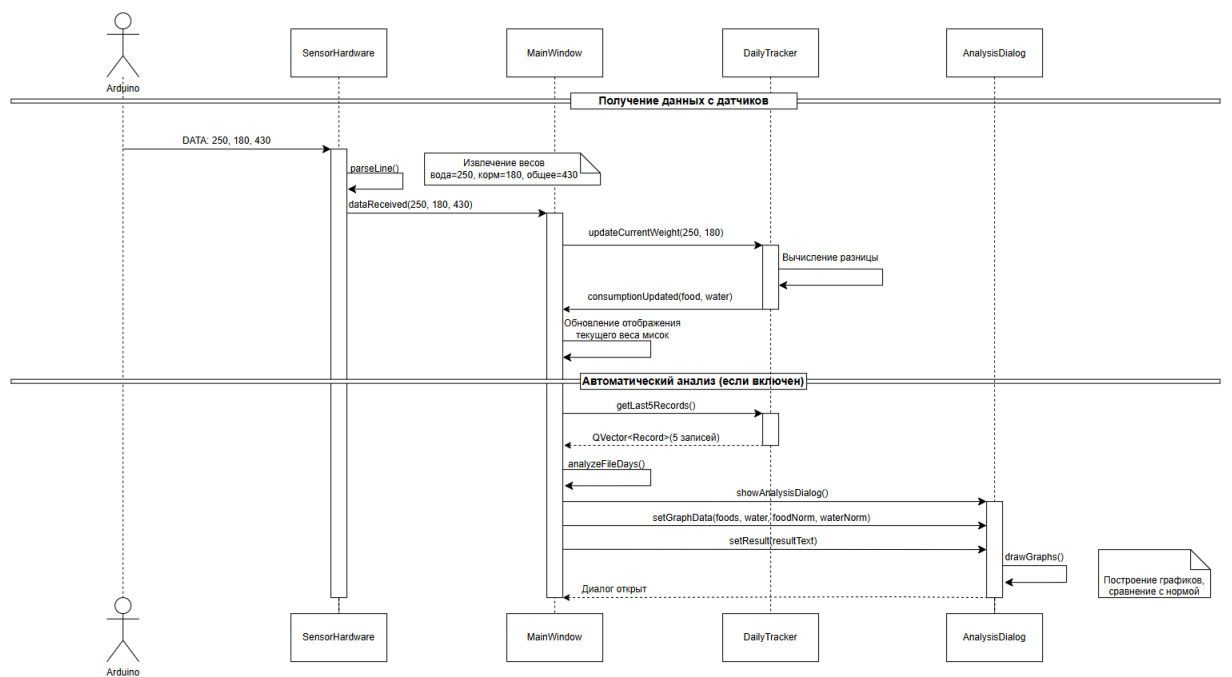


Рисунок 2 - Диаграмма последовательности

Пояснение шагов диаграммы последовательности:

1. Arduino отправляет строку `DATA:250,180,430`. `SensorHardware` получает её в потоке чтения.
2. `SensorHardware` вызывает `parseLine(line)`, которая извлекает три числа: вес воды (250 г), вес корма (180 г), общий вес (430 г). Данные сохраняются под мьютексом.

3. SensorHardware испускает сигнал dataReceived(250, 180, 430). Этот сигнал подключён к слоту MainWindow onSensorDataReceived.
4. MainWindow вызывает m_tracker->updateCurrentWeight(250, 180), передавая текущие веса.
5. DailyTracker вычисляет разницу между предыдущим и текущим весом. Если разница положительная (питомец поел или попил), она добавляется к счётчикам потребления.
6. DailyTracker испускает сигнал consumptionUpdated(food, water), который обновляет отображение съеденного и выпитого в интерфейсе.
7. MainWindow обновляет панель с текущим весом мисок
8. Если включён автоматический анализ, MainWindow вызывает getLast5Records() для получения данных за последние пять дней.
9. DailyTracker возвращает вектор из пяти последних записей.
10. MainWindow вызывает analyzeFiveDays() с полученными данными.
11. MainWindow создаёт диалог AnalysisDialog и вызывает showAnalysisDialog().
12. MainWindow вызывает setGraphData() для передачи векторов потребления и норм.
13. MainWindow вызывает setResult() для передачи текстовых рекомендаций.
14. AnalysisDialog строит графики и отображает их, а также показывает текстовые рекомендации владельцу.

Диаграмма показывает полный путь данных: от физического измерения на Arduino через передачу данных по Bluetooth или по USB-порт, парсинг в SensorHardware, передачу в MainWindow и DailyTracker, до визуализации в AnalysisDialog. Она демонстрирует слабосвязанность компонентов через сигнально-слотовую модель: SensorHardware не знает о существовании

MainWindow, DailyTracker не знает о AnalysisDialog — они просто испускают сигналы, на которые подписаны другие объекты.

2.2 Реализация графического интерфейса

Графический интерфейс приложения реализован с использованием Qt Designer. Главное окно MainWindow наследуется от QMainWindow и использует класс Ui::MainWindow, сгенерированный из файла mainwindow.ui.

На рисунке 3 представлен конструктор MainWindow. В нём выполняются следующие действия: установка пользовательского интерфейса через `ui->setupUi(this)`, инициализация справочных данных о породах (`initData()`), инициализация датчиков и трекера (`initSensors()`), добавление элементов в выпадающий список видов животных (`dog, cat, rabbit`), подключение сигналов от виджетов к соответствующим слотам.

```
MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent), ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    initData();
    initSensors();

    ui->speciesCombo->addItem("dog", "cat", "rabbit");

    connect(ui->speciesCombo, QOverload<int>::of(&QComboBox::currentIndexChanged),
        this, &MainWindow::onSpeciesChanged);
    connect(ui->breedCombo, QOverload<int>::of(&QComboBox::currentIndexChanged),
        this, &MainWindow::onBreedChanged);
    connect(ui->analyzeBtn, &QPushButton::clicked,
        this, &MainWindow::onAnalyzeClicked);
}
```

Рисунок 3 - Конструктор MainWindow

На рисунке 4 показана настройка таблицы, в которую владелец вручную вводит данные о потреблении за 5 дней (если датчики отключены). Таблица имеет 5 строк и 2 столбца (корм в граммах и вода в миллилитрах). Ячейки изначально заполняются символом «—», делаются нередактируемыми и центрируются по тексту. `setFlags(item->flags() & ~Qt::ItemIsEditable)` удаляет флаг редактируемости, оставляя все остальные флаги неизменными. Это гарантирует, что в режиме автоматического сбора данных владелец случайно не изменит значения.

```

// Настройка таблицы 5 дней
ui->tableWidget->setRowCount(5);
ui->tableWidget->setColumnCount(2);
ui->tableWidget->setHorizontalHeaderLabels({"Корм (г)", "Вода (мл)"});
for (int row = 0; row < 5; ++row) {
    QTableWidgetItem *itemFood = new QTableWidgetItem("-");
    QTableWidgetItem *itemWater = new QTableWidgetItem("-");
    itemFood->setTextAlignment(Qt::AlignCenter);
    itemWater->setTextAlignment(Qt::AlignCenter);
    itemFood->setFlags(itemFood->flags() & ~Qt::ItemIsEditable);
    itemWater->setFlags(itemWater->flags() & ~Qt::ItemIsEditable);
    ui->tableWidget->setItem(row, 0, itemFood);
    ui->tableWidget->setItem(row, 1, itemWater);
}
ui->tableWidget->setColumnWidth(0, 100);
ui->tableWidget->setColumnWidth(1, 100);

```

Рисунок 4 - Настройка таблицы для ввода данных

На рисунке 5 показано заполнение списка заболеваний. Для каждого заболевания создаётся элемент `QListWidgetItem`, устанавливается русское название (отображается пользователю), а в `UserRole` сохраняется английский идентификатор (используется в анализе). Элемент делается чекбоксом с помощью `Qt::ItemIsUserCheckable`. `setData(Qt::UserRole, it.key())` сохраняет пользовательские данные в элементе. При анализе программа читает `UserRole`, чтобы получить английский идентификатор заболевания, и проверяет `checkState()`, чтобы узнать, отмечено ли оно.

```

// Заболевания
QMap<QString, QString> diseaseMap;
diseaseMap["allergy"] = "Аллергия";
diseaseMap["arthritis"] = "Артрит";
diseaseMap["diabetes"] = "Диабет";
diseaseMap["kidney_disease"] = "Болезнь почек";
diseaseMap["obesity"] = "Ожирение";
diseaseMap["underweight"] = "Недостаточный вес";

for (auto it = diseaseMap.begin(); it != diseaseMap.end(); ++it) {
    QListWidgetItem *item = new QListWidgetItem(it.value());
    item->setData(Qt::UserRole, it.key());
    item->setFlags(item->flags() | Qt::ItemIsUserCheckable);
    item->setCheckState(Qt::Unchecked);
    ui->diseaseList->addItem(item);
}

```

Рисунок 5 - Заполнение списка заболеваний

На рисунке 6 показан метод `addLogMessage()`, который добавляет сообщение в лог СОМ-порта. Он добавляет временную метку в формате «чч:мм:сс», затем автоматически прокручивает лог вниз, чтобы владелец всегда видел последние сообщения.

```
// ===== ЛОГ СО М-ПОРТА =====
void MainWindow::addLogMessage(const QString& message)
{
    QString timestamp = QDateTime::currentDateTime().toString("hh:mm:ss");
    QString logEntry = QString("[%1] %2").arg(timestamp, message);

    ui->comLogTextEdit->append(logEntry);

    QScrollBar* sb = ui->comLogTextEdit->verticalScrollBar();
    sb->setValue(sb->maximum());
}
```

Рисунок 6 - Метод `addLogMessage()`

На рисунке 7 показан метод `setTableEditMode()`, который включает или выключает редактирование таблицы. В автоматическом режиме (датчики подключены) ячейки становятся нередактируемыми и окрашиваются в зелёный цвет. В ручном режиме ячейки редактируемы и имеют чёрный цвет.

```
// ===== ПЕРЕКЛЮЧЕНИЕ РЕЖИМА ТАБЛИЦЫ =====
void MainWindow::setTableEditMode(bool editable)
{
    m_manualInputMode = editable;

    for (int row = 0; row < ui->tableWidget->rowCount(); ++row) {
        for (int col = 0; col < ui->tableWidget->columnCount(); ++col) {
            QTableWidgetItem* item = ui->tableWidget->item(row, col);
            if (!item) continue;

            if (editable) {
                item->setFlags(item->flags() | Qt::ItemIsEditable);
                item->setForeground(Qt::black);
            } else {
                item->setFlags(item->flags() & ~Qt::ItemIsEditable);
                item->setForeground(QColor(0, 100, 0));
            }
        }
    }

    qDebug() << "Режим таблицы:" << (editable ? "Ручной ввод" : "Данные с датчиков");
}
```

Рисунок 7 - Метод `setTableEditMode()`

2.3 Реализация модуля ежедневного трекинга

Класс DailyTracker реализует автоматический учёт потребления воды и корма. Он использует таймер для периодического сохранения записей и вычисляет потребление как разницу между начальным и текущим весом миски.

На рисунке 8 представлен заголовочный файл класса DailyTracker. Показаны основные атрибуты: таймер сохранения, начальные и текущие веса, счётчики потребления, флаг инициализации и вектор истории записей. Также показаны сигналы recordSaved, historyUpdated и consumptionUpdated.

```
class DailyTracker : public QObject
{
    Q_OBJECT

public:
    explicit DailyTracker(QObject *parent = nullptr);

    void setSaveInterval(int seconds);
    void startTracking();
    void stopTracking();

    struct Record {
        QDateTime timestamp;
        double foodConsumed;
        double waterConsumed;
    };

    QVector<Record> getLast5Records() const { return m_history; }

signals:
    void recordSaved(const QDateTime& timestamp, double foodConsumed, double waterConsumed);
    void historyUpdated(const QVector<DailyTracker::Record>& records);
    void consumptionUpdated(double food_g, double water_ml);

private slots:
    void onSaveTimer();

private:
    QTimer* m_saveTimer = nullptr;
    int m_saveInterval = 30; // секунды

    double m_startWater = 0.0;
    double m_startFood = 0.0;

    double m_currentWater = 0.0;
    double m_currentFood = 0.0;

    double m_consumedWater = 0.0;
    double m_consumedFood = 0.0;

    bool m_initialized = false;

    QVector<Record> m_history;
```

Рисунок 8 - Заголовочный файл класса DailyTracker

На рисунке 9 показан конструктор DailyTracker. Он создаёт таймер, устанавливает тип таймера Qt::VeryCoarseTimer (оптимизирован для интервалов более 1 секунды) и подключает сигнал timeout к слоту onSaveTimer.

```
DailyTracker::DailyTracker(QObject *parent)
: QObject(parent)
{
    m_saveTimer = new QTimer(this);
    m_saveTimer->setTimerType(Qt::VeryCoarseTimer); // для интервалов > 1 сек
    connect(m_saveTimer, &QTimer::timeout, this, &DailyTracker::onSaveTimer);

    qDebug() << "DailyTracker создан. Запуск таймера с интервалом:" << m_saveInterval << "сек";
}
```

Рисунок 9 - Конструктор DailyTracker

На рисунке 10 показан метод updateCurrentWeight(), который вызывается при получении новых данных от датчиков. При первом вызове фиксируются начальные веса. Затем вычисляется разница между начальным и текущим весом — это и есть потребление. Если разница отрицательная (обнаружено пополнение миски), начальный вес сбрасывается на текущий.

```
void DailyTracker::updateCurrentWeight(double water_ml, double food_g)
{
    m_currentWater = water_ml;
    m_currentFood = food_g;

    // При первом получении данных фиксируем начальные веса и запускаем таймер
    if (!m_initialized) {
        m_startWater = water_ml;
        m_startFood = food_g;
        m_initialized = true;

        // Запускаем таймер при первом получении данных
        if (!m_saveTimer->isActive()) {
            m_saveTimer->start(m_saveInterval * 1000);
            qDebug() << "🟢 Трекер инициализирован и таймер запущен. Вода:" << m_startWater << "г, Корм:" << m_startFood << "г";
        }

        emit consumptionUpdated(0, 0);
        return;
    }

    // Вычисляем потребление с момента последнего сохранения
    double waterConsumed = m_startWater - m_currentWater;
    double foodConsumed = m_startFood - m_currentFood;

    // Защита от отрицательных значений (если долили/досыпали)
    if (waterConsumed < 0) {
        m_startWater = m_currentWater;
        waterConsumed = 0;
        qDebug() << "💧 Обнаружено пополнение воды. Сброс начального веса.";
    }
    if (foodConsumed < 0) {
        m_startFood = m_currentFood;
        foodConsumed = 0;
        qDebug() << "🍲 Обнаружено пополнение корма. Сброс начального веса.";
    }

    m_consumedWater = waterConsumed;
    m_consumedFood = foodConsumed;

    qDebug() << "📊 Текущее потребление: Корм" << m_consumedFood << "г, Вода" << m_consumedWater << "мл";
    emit consumptionUpdated(m_consumedFood, m_consumedWater);
}
```

Рисунок 10 - Метод updateCurrentWeight()

На рисунке 11 показан слот `onSaveTimer()`, который вызывается каждые 30 секунд. Он сохраняет текущую запись (даже если потребление равно нулю, чтобы в истории был непрерывный ряд), затем сбрасывает начальные веса на текущие и обнуляет счётчики потребления.

```
void DailyTracker::onSaveTimer()
{
    qDebug() << "🕒 ТАЙМЕР СРАБОТАЛ! initialized =" << m_initialized
              << "consumedFood =" << m_consumedFood << "consumedWater =" << m_consumedWater;

    if (!m_initialized) {
        qDebug() << "❌ Трекер не инициализирован — пропускаем сохранение";
        return;
    }

    // Сохраняем, даже если потребление = 0 (чтобы было видно в таблице)
    saveCurrentRecord();

    // Сбрасываем начальные веса на текущие для следующего интервала
    m_startWater = m_currentWater;
    m_startFood  = m_currentFood;
    m_consumedWater = 0;
    m_consumedFood  = 0;

    qDebug() << "🔄 Начальные веса сброшены. Таймер перезапущен.";
}
```

Рисунок 11 - Слот `onSaveTimer()`

На рисунке 12 показан метод `saveCurrentRecord()`. Он создаёт структуру `Record`, заполняет её текущей датой и временем, а также накопленным потреблением, затем вызывает `updateHistory()`. `QDateTime::currentDateTime()` возвращает текущую дату и время с точностью до миллисекунды.

```
void DailyTracker::saveCurrentRecord()
{
    Record record;
    record.timestamp = QDateTime::currentDateTime();
    record.foodConsumed = m_consumedFood;
    record.waterConsumed = m_consumedWater;

    qDebug() << "💾 СОХРАНЕНИЕ ЗАПИСИ:"
              << record.timestamp.toString("hh:mm:ss")
              << "| Корм:" << record.foodConsumed << "г"
              << "| Вода:" << record.waterConsumed << "мл";

    updateHistory(record);

    emit recordSaved(record.timestamp, record.foodConsumed, record.waterConsumed);
}
```

Рисунок 12 - Метод `saveCurrentRecord()`

На рисунке 13 показан метод `updateHistory()`. Он добавляет запись в вектор `m_history`, сортирует записи по времени, затем удаляет самые старые записи, оставляя только последние 5.

```
void DailyTracker::updateHistory(const Record& record)
{
    m_history.append(record);

    std::sort(m_history.begin(), m_history.end(),
        [](const Record& a, const Record& b) { return a.timestamp < b.timestamp; });

    while (m_history.size() > 5) {
        m_history.removeFirst();
    }

    qDebug() << "📅 История обновлена. Всего записей:" << m_history.size();

    for (int i = 0; i < m_history.size(); ++i) {
        qDebug() << "  [" << i << "]" << m_history[i].timestamp.toString("hh:mm:ss")
            << "Корм:" << m_history[i].foodConsumed << "Вода:" << m_history[i].waterConsumed;
    }

    emit historyUpdated(m_history);
}
```

Рисунок 13 - Метод `updateHistory()`

На рисунке 14 показан метод `initSensors()`, который создаёт объект `DailyTracker`, устанавливает интервал сохранения 30 секунд и подключает его сигналы к слотам `MainWindow`.

```
// ===== ИНИЦИАЛИЗАЦИЯ ДАТЧИКОВ =====
void MainWindow::initSensors()
{
    m_tracker = new DailyTracker(this);
    m_tracker->setSaveInterval(30); // 30 секунд – теперь запускает таймер

    connect(m_tracker, &DailyTracker::recordSaved,
        this, &MainWindow::onRecordSaved);
    connect(m_tracker, &DailyTracker::historyUpdated,
        this, &MainWindow::onHistoryUpdated);
    connect(m_tracker, &DailyTracker::consumptionUpdated,
        this, &MainWindow::onConsumptionUpdated);

    m_autoAnalyzeTimer = new QTimer(this);
    connect(m_autoAnalyzeTimer, &QTimer::timeout,
        this, &MainWindow::onAutoAnalyzeCheck);
    m_autoAnalyzeTimer->start(600000);

    qDebug() << "Система датчиков инициализирована (интервал сохранения: 30 сек)";
    addLogMessage("Трекер инициализирован. Ожидание данных с датчиков...");
}
```

Рисунок 14 - Метод `initSensors()`

На рисунке 15 показан слот `onSensorDataReceived()`, который вызывается при получении новых данных от `SensorHardware`. Он передаёт вес воды и корма в трекер через `updateCurrentWeight()`.

```
// ===== ДАННЫЕ С ДАТЧИКОВ =====  
void MainWindow::onSensorDataReceived(double water, double food, double total)  
{  
    m_tracker->updateCurrentWeight(water, food);  
    qDebug() << "Датчики: вода =" << water << "г, корм =" << food  
        << "г, всего =" << total << "г";  
}
```

Рисунок 15 - Слот `onSensorDataReceived()`

На рисунке 16 показан метод `forceSave()`, который вызывается при отключении датчиков или завершении программы. Он сохраняет текущую запись и сбрасывает счётчики.

```
void DailyTracker::forceSave()  
{  
    qDebug() << "⚡ Принудительное сохранение";  
    if (!m_initialized) {  
        qDebug() << "❌ Трекер не инициализирован";  
        return;  
    }  
    saveCurrentRecord();  
  
    // Сбрасываем начальные веса  
    m_startWater = m_currentWater;  
    m_startFood = m_currentFood;  
    m_consumedWater = 0;  
    m_consumedFood = 0;  
}
```

Рисунок 16 - Метод `forceSave()`

2.4 Реализация аппаратного модуля и связи с ним

Класс `SensorHardware` отвечает за связь с аппаратным модулем на Arduino через последовательный порт. Он использует Windows API для работы с COM-портом и запускает отдельный поток для чтения данных.

На рисунке 17 представлен заголовочный файл `SensorHardware`. Показаны атрибуты: имя порта, хендл порта, атомарные флаги для потокобезопасности, мьютекс для защиты данных и переменные для хранения

весов. Также показаны сигналы для передачи данных в MainWindow. `std::atomic<bool>` обеспечивает атомарные операции чтения/записи, что гарантирует безопасность при доступе к флагам из разных потоков. `QMutex` защищает доступ к весам, так как они читаются из основного потока и записываются из потока чтения.

```
class SensorHardware : public QObject
{
    Q_OBJECT

public:
    explicit SensorHardware(const QString& portName, QObject *parent = nullptr);
    ~SensorHardware() override;

    bool start();
    void stop();
    void sendCommand(const QString& cmd);

signals:
    void dataReceived(double water_g, double food_g, double total_g);
    void connectionStatusChanged(bool connected, const QString& status);
    void errorOccurred(const QString& error);
    void rawDataReceived(const QString& rawLine);

private:
    QString m_portName;
    HANDLE m_hSerial = INVALID_HANDLE_VALUE;
    std::atomic<bool> m_running{ false };
    std::atomic<bool> m_connected{ false };
    std::atomic<bool> m_hasData{ false };

    mutable QMutex m_dataMutex;
    double m_waterWeight = 0.0;
    double m_foodWeight = 0.0;
    double m_totalWeight = 0.0;
};
```

Рисунок 17 - Заголовочный файл SensorHardware

На рисунке 18 показан метод `openSerial()`, который открывает COM-порт с параметрами 9600 бод, 8 бит данных, 1 стоп-бит, без чётности. Используется Windows API: `CreateFileA` для открытия порта, `GetCommState` и `SetCommState` для настройки параметров.

```

bool SensorHardware::openSerial()
{
    QString fullPath = "\\.\\" + m_portName;

    m_hSerial = CreateFileA(
        fullPath.toStdString().c_str(),
        GENERIC_READ | GENERIC_WRITE,
        0, nullptr, OPEN_EXISTING,
        FILE_ATTRIBUTE_NORMAL, nullptr
    );

    if (m_hSerial == INVALID_HANDLE_VALUE) { ... }

    DCB dcb = { sizeof(DCB) };
    if (!GetCommState(m_hSerial, &dcb)) { ... }

    dcb.BaudRate = CBR_9600;
    dcb.ByteSize = 8;
    dcb.Parity = NOPARITY;
    dcb.StopBits = ONESTOPBIT;

    if (!SetCommState(m_hSerial, &dcb)) { ... }

    SetCommTimeouts(m_hSerial, &timeouts);

    m_connected = true;

    return true;
}

```

Рисунок 18 - Метод openSerial()

На рисунке 19 показан метод readLoop(), который выполняется в отдельном потоке. Он циклически читает данные из порта, накапливает их до символа перевода строки и передаёт строку в parseLine().

```

void SensorHardware::readLoop()
{
    qDebug() << "readLoop запущен. Ожидание данных...";

    char buffer[256];
    QByteArray lineBuffer;

    while (m_running) {
        DWORD bytesRead = 0;
        BOOL result = ReadFile(m_hSerial, buffer, sizeof(buffer) - 1, &bytesRead, nullptr);

        if (!result) { ... }

        if (bytesRead > 0) {
            buffer[bytesRead] = '\0';
            lineBuffer.append(buffer);

            qDebug() << "Прочитано" << bytesRead << "байт:" << QByteArray(buffer, bytesRead).toHex();

            int pos;
            while ((pos = lineBuffer.indexOf('\n')) != -1) { ... }
        }
        QThread::msleep(10);
    }

    qDebug() << "readLoop завершён";
}

```

Рисунок 19 - Метод readLoop()

На рисунке 20 показан метод `parseLine()`, который разбирает строки, полученные с Arduino. Ожидается формат DATA:вес_воды,вес_корма,общий_вес. Данные сохраняются под мьютексом, и испускается сигнал `dataReceived`.

```
void SensorHardware::parseLine(const QString& line)
{
    qDebug() << "parseLine:" << line;

    if (!line.startsWith("DATA:")) { ... }

    QString data = line.mid(5);
    QStringList parts = data.split(',');

    qDebug() << " -> parts:" << parts;

    if (parts.size() >= 2) {
        bool ok1, ok2, ok3 = false;
        double w = parts[0].toDouble(&ok1);
        double f = parts[1].toDouble(&ok2);
        double t = (parts.size() >= 3) ? parts[2].toDouble(&ok3) : (w + f);

        qDebug() << " -> w=" << w << "f=" << f << "t=" << t << "ok1=" << ok1 << "ok2=" << ok2;

        if (ok1 && ok2) { ... }
    }
}
```

Рисунок 20 - Метод `parseLine()`

На рисунке 21 показан метод `sendCommand()`, который отправляет команду на Arduino. Команда дополняется символом перевода строки и записывается в порт через `WriteFile`.

```
void SensorHardware::sendCommand(const QString& cmd)
{
    if (m_hSerial == INVALID_HANDLE_VALUE) {
        qDebug() << "Невозможно отправить команду — порт не открыт";
        return;
    }

    QByteArray data = (cmd + "\n").toUtf8();
    DWORD written = 0;
    WriteFile(m_hSerial, data.constData(), data.size(), &written, nullptr);

    QString timestamp = QDateTime::currentDateTime().toString("hh:mm:ss.zzz");
    emit rawDataReceived(QString("[%1] TX: %2").arg(timestamp, cmd));
    qDebug() << "Отправлена команда:" << cmd;
}
```

Рисунок 21 - Метод `sendCommand()`

На рисунке 22 показан слот `onDisconnectSensor()`, который останавливает датчики, удаляет объект и переключает интерфейс в режим ручного ввода.

```
void MainWindow::onDisconnectSensor()
{
    if (m_sensor) {
        addLogMessage("Отключение от " + m_sensor->portName() + "...");
        m_sensor->stop();
        delete m_sensor;
        m_sensor = nullptr;
    }
    m_sensorEnabled = false;

    setTableEditMode(true);

    ui->sensorStatusLabel->setText("Статус: Датчики не подключены (ручной ввод)");
    ui->sensorStatusLabel->setStyleSheet("color: gray; font-style: italic;");

    addLogMessage("Датчики отключены. Режим ручного ввода.");
    qDebug() << "Датчики отключены. Режим ручного ввода активирован.";
}
```

Рисунок 22 - Слот `onDisconnectSensor()`

2.5 Реализация и визуализация

Анализ выполняется в `MainWindow`, а визуализация — в отдельном диалоге `AnalysisDialog`.

На рисунке 23 показана структура `BreedNorm`, которая хранит нормы для конкретной породы: русское название, средние нормы корма и воды, минимальный и максимальный вес, минимальный и максимальный рост.

```
struct BreedNorm {
    QString nameRus;
    double avgFood;
    double avgWater;
    double weightMin;
    double weightMax;
    double heightMin;
    double heightMax;
};
```

Рисунок 23 - Структура `BreedNorm`

На рисунке 24 показана инициализация справочных данных для пород в методе `initData()`.

```
QMap<QString, BreedNorm> dogBreeds;
dogBreeds["labrador"] = {"Лабрадор", 400, 650, 25, 36, 55, 62};
dogBreeds["german_shepherd"] = {"Немецкая овчарка", 450, 750, 30, 40, 55, 65};
dogBreeds["bulldog"] = {"Бульдог", 320, 500, 20, 25, 30, 40};
dogBreeds["poodle"] = {"Пудель", 250, 450, 8, 25, 25, 60};
dogBreeds["chihuahua"] = {"Чихуахуа", 75, 150, 1, 3, 15, 25};
dogBreeds["husky"] = {"Хаски", 400, 650, 20, 27, 50, 60};
dogBreeds["dachshund"] = {"Такса", 220, 350, 7, 14, 20, 27};
dogBreeds["rottweiler"] = {"Ротвейлер", 500, 850, 38, 50, 56, 69};
dogBreeds["beagle"] = {"Бигль", 240, 400, 9, 11, 33, 41};
dogBreeds["boxer"] = {"Боксёр", 420, 680, 25, 32, 53, 63};
dogBreeds["mongrel"] = {"Беспородная собака", 350, 550, 10, 35, 25, 60};
speciesBreeds["dog"] = dogBreeds;
```

Рисунок 24 - Метод `initData()`

На рисунке 25 показан метод `analyzeFiveDays()`, который анализирует потребление корма и воды за 5 дней. Сначала проверяются серьёзные заболевания — при их наличии сразу выдаётся критическое предупреждение. Затем для каждого из 5 дней вычисляется отношение фактического потребления к норме.

```
void MainWindow::showAnalysisDialog(const QString& species, const QString& breed,
                                   double weight, double age,
                                   const QStringList& diseases,
                                   const QVector<double>& foods,
                                   const QVector<double>& waters)
{
    BreedNorm norm = getBreedNorm(species, breed);

    AnalysisDialog* dialog = new AnalysisDialog(this);

    QString title = QString("<b>%1 (%2)</b><br>"
                           "Вес: %3 кг | Возраст: %4 лет")
        .arg(norm.nameRus)
        .arg(species)
        .arg(weight)
        .arg(age);
    dialog->setTitle(title);

    QString result = analyzeFiveDays(species, breed, weight, age, diseases, foods, waters);

    QString htmlResult;
    if (result.startsWith("❗")) {  }
    } else if (result.startsWith("⚠️")) {  }
    } else if (result.startsWith("📋")) {  }
    } else if (result.startsWith("✅")) {  }
    } else {  }
```

Рисунок 25 - Метод `analyzeFiveDays()`

На рисунке 26 показан метод drawGraphs() из класса AnalysisDialog, который строит графики с использованием QPainter.

```
// ===== ГРАФИКИ =====  
void MainWindow::drawGraphs(const QVector<double> &foods, const QVector<double> &waters,  
                             double foodNorm, double waterNorm)  
{  
    if (ui->foodGraphLabel->width() < 10 || ui->foodGraphLabel->height() < 10) { ... }  
    if (ui->waterGraphLabel->width() < 10 || ui->waterGraphLabel->height() < 10) { ... }  
  
    QPixmap foodPix(ui->foodGraphLabel->width(), ui->foodGraphLabel->height());  
    QPixmap waterPix(ui->waterGraphLabel->width(), ui->waterGraphLabel->height());  
    foodPix.fill(Qt::white);  
    waterPix.fill(Qt::white);  
  
    QPainter pFood(&foodPix);  
    QPainter pWater(&waterPix);  
    pFood.setRenderHint(QPainter::Antialiasing);  
    pWater.setRenderHint(QPainter::Antialiasing);  
}
```

Рисунок 26 - Метод drawGraphs()

Вывод по разделу 2. В практической части реализованы все компоненты АРМ. Графический интерфейс содержит таблицу для ручного ввода данных за 5 дней, список заболеваний, лог СОМ-порта. Класс DailyTracker с помощью таймера (интервал 30 секунд) автоматически сохраняет записи и отслеживает изменение веса мисок, корректно обрабатывая пополнение мисок. Класс SensorHardware работает с последовательным портом через Windows API, использует отдельный поток для чтения, атомарные переменные и мьютекс для потокобезопасности, парсит строки формата DATA. Класс AnalysisDialog отображает графики потребления с нанесением линии нормы и текстовые рекомендации. Анализ данных использует пороги 50%, 80%, 120%, 150% от нормы и учитывает серьёзные заболевания.

ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта разработано автоматизированное рабочее место хозяина по уходу за домашним питомцем. Проект включает программную часть на языке C++ с использованием фреймворка Qt и аппаратный модуль на базе микроконтроллера Arduino с двумя тензодатчиками, усилителями HX711 и Bluetooth-модулем HC-05.

В теоретической части проведён анализ предметной области, рассмотрены зоотехнические нормы содержания питомцев. Описаны физические принципы работы тензодатчиков и усилителя HX711. Проведён анализ существующих решений, показавший их высокую стоимость или недостаточную функциональность.

В практической части реализованы все компоненты АРМ. Графический интерфейс содержит все требуемые виджеты: таблицу для ввода данных, дерево пород, выпадающие списки, поля ввода, список заболеваний, лог COM-порта. Разработан класс DailyTracker с таймером автоматического сохранения и алгоритмом отслеживания потребления. Реализован класс SensorHardware для связи с Arduino через последовательный порт. Создан диалог AnalysisDialog для отображения графиков потребления и текстовых рекомендаций.

Все поставленные задачи выполнены. Разработанное АРМ позволяет владельцу автоматически контролировать питание питомца, получать отчёты о съеденном корме и выпитой воде за 5 дней, а также сравнивать показатели с нормами для конкретной породы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. <https://www.proplan.ru/vet/article/gidrataciya-koshek-i-sobak>
2. <https://vetson.ru/info/publications/statyi/vazhnost-vody-skolko-dolzhen-pit-pitomets-v-sutki/>
3. <https://kovcheg.vet/koshki-sobaki-uhod-korm/kak-rasschitat-sutochnuyu-normu-korma-dlya-koshki-i-sobaki/>
4. <https://sierra.market/company/articles/187/>
5. <https://3d-diy.ru/blog/acp-dlya-analogovyx-vesov-hx711/>
6. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ, 2017.
7. Официальная документация Qt 6. – URL: <https://doc.qt.io/qt-6/>
8. Алексеева Е. Р., Злобина Г. Г., Костюк Д. А., Чеснокова О. В., Чмыхало А. С. Программирование на языке C++ в среде Qt Creator. – М.: ДМК Пресс, 2021. – 356 с.